Department of Electrical and Computer Engineering
Faculty of Engineering & Technology
International Islamic University, Islamabad

# CS100L

# Fundamentals of Programming

## Lab Manual

| | |
|---|---|
| Subject Teacher: | |
| Lab Instructor: | |

| | |
|---|---|
| Student Name | |
| Reg. No. | |
| Section | |

**Prepared by:**

Engr. Rashid Farid Chishti
Lecturer, Department of Electrical and Computer Engineering.
Faculty of Engineering and Technology.
International Islamic University, Islamabad.

# OBJECTIVE

This lab has been designed for students with no prior programming experience. This lab provides hands on experience on programming in C++. It introduces the fundamental concepts of procedural programming. Topics include data types, control structures, functions, arrays, files, and the mechanics of running, testing, and debugging.

| CLO | CLO Description | DOMAIN | PLO |
|---|---|---|---|
| **01** | **Design** codes to solve simple problems. | C5 | PLO2 |
| **02** | Do programming **practice** using a compiler | P3 | PLO3 |
| **03** | **Participate** effectively as an individual or in a team | A2 | PLO9 |

CLO: Class Learning Outcome.

PLO: Program Learning Outcome.

LIST OF EXPERIMENTS

**LAB 1 : INTRODUCTION TO DEV-C++. 1**

**LAB 2 : ESCAPE SEQUENCE. 6**

**LAB 3 : USER INPUT AND OUTPUT. 11**

**LAB 4 : FORMATTED OUTPUT. 19**

**LAB 5 : BITWISE OPERATORS. 25**

**LAB 6 : LOGIC OPERATORS. 31**

**LAB 7 : DECISIONS AND LOOPS. 38**

**LAB 8 : DO…WHILE AND FOR LOOP. 46**

**LAB 9 : FUNCTIONS. 53**

**LAB 10 : INLINE AND RECURSIVE FUNCTIONS. 57**

**LAB 11 : ARRAYS. 62**

**LAB 12 : STRING 68**

**LAB 13 : STRUCTURE. 73**

**LAB 14 : POINTERS. 79**

**LAB 15 FILE HANDLING. 84**

# Lab 1 : Introduction to Dev-C++.

## Objective:

The objective of the lab is to understand how to do C++ programming using DEV-C++ compiler.

## DEV-C++ Compiler:

First of all, we will discuss how to download, install, and work on Dev-C++ compiler:

### Downloading:

You can download the compiler from course web site using following link.

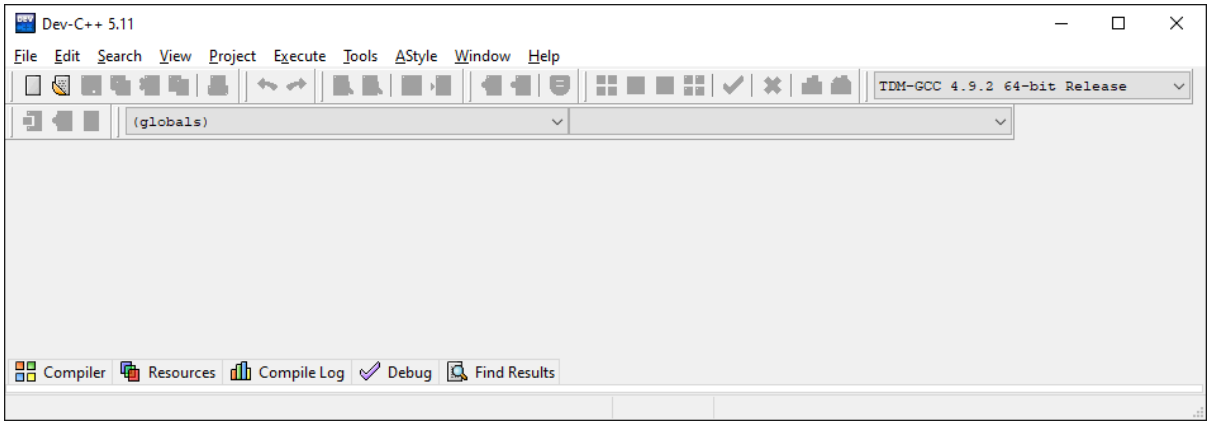
*https://sites.google.com/site/chishti/home/intro_cpp*

### Installation:

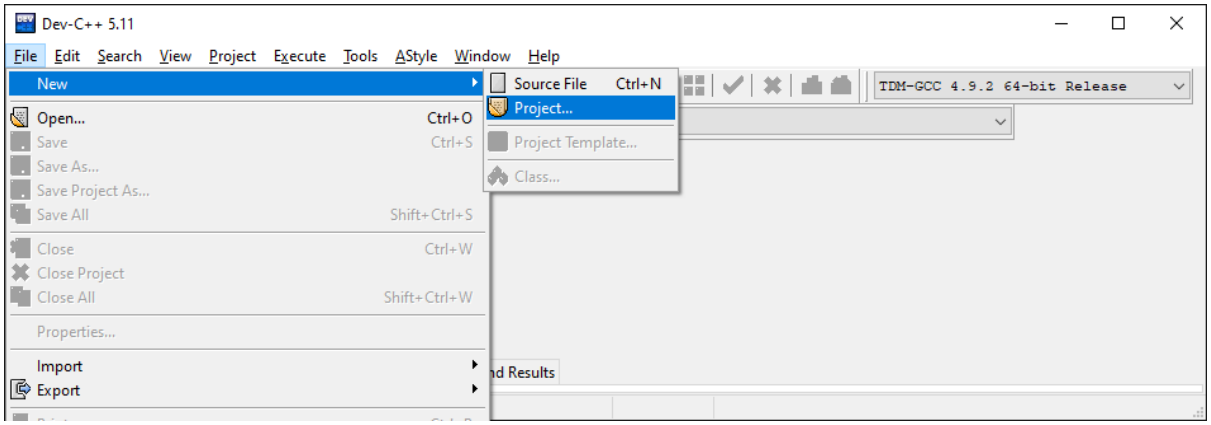
double clock on the downloaded file and install it on computer and follow the instruction.

### Working on DEV-C++:

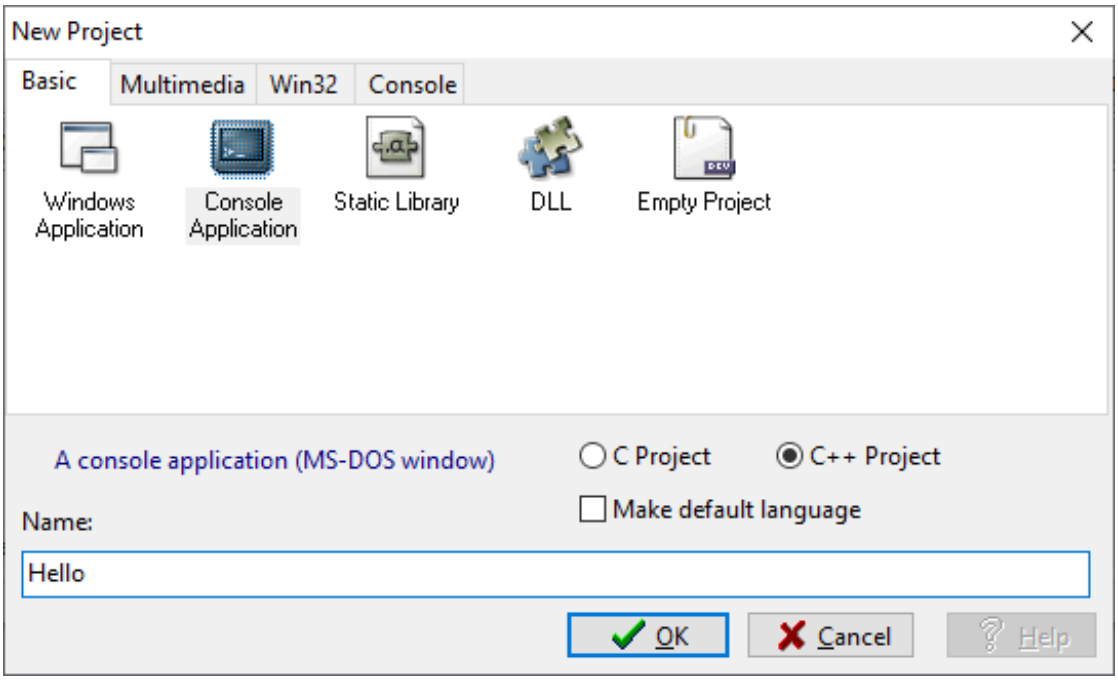
After installation you will see a shortcut named **Dev-C++** on your computer. Double click on the shortcut you will have following screen.



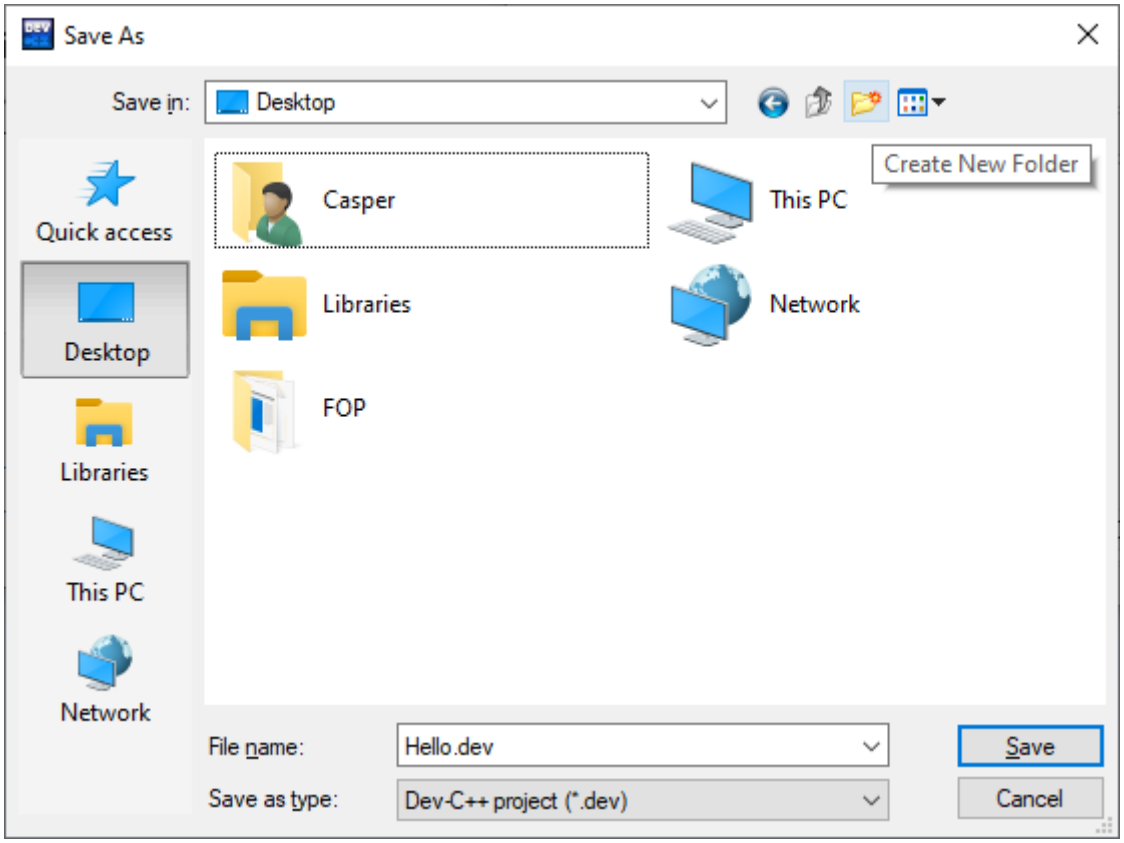

Click on **File** > **New** > **Project**



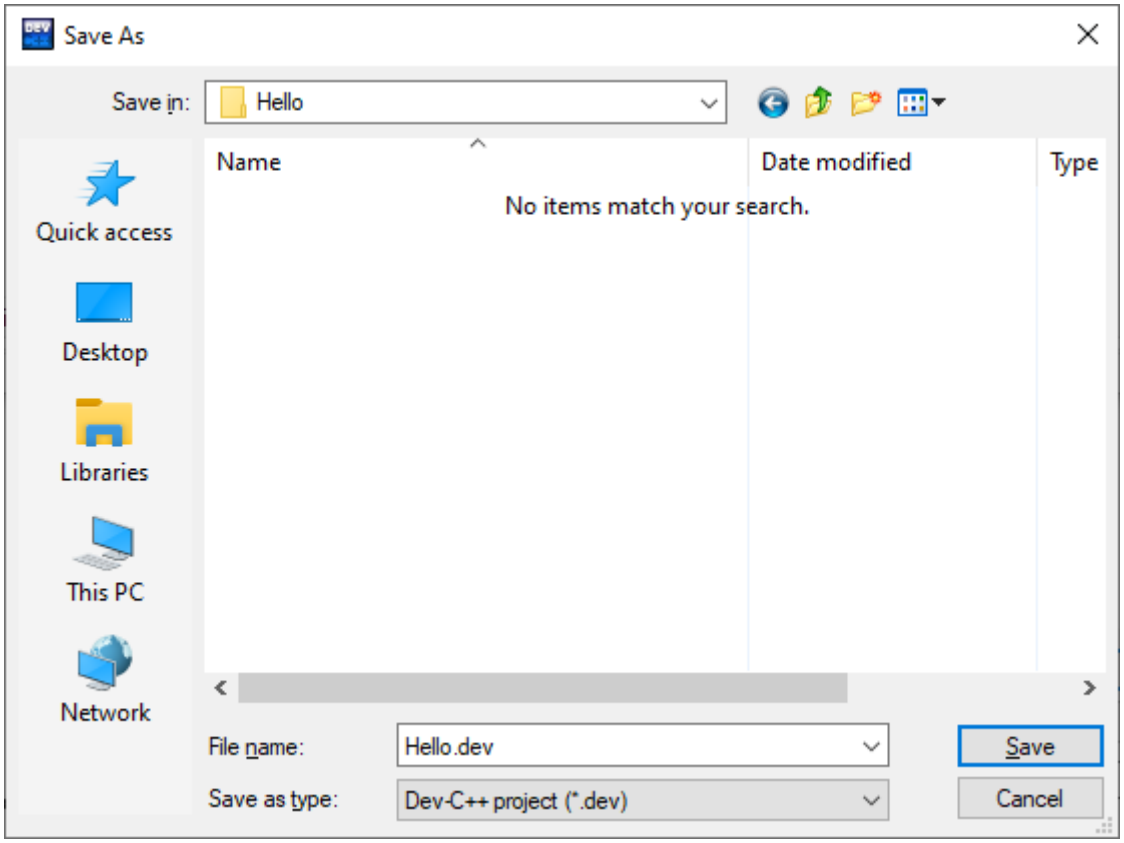
Click on **Console Application** and Write project name as **Hello** and press **OK** Button.




Now click on **Desktop** icon to go to desktop. After that click on **Create New Folder** icon and write folder name as **Hello**.




Double Click on folder named **Hello** that you just made. Write project name as **Hello.dev** and save project inside the folder **Hello** by pressing **Save** Button.

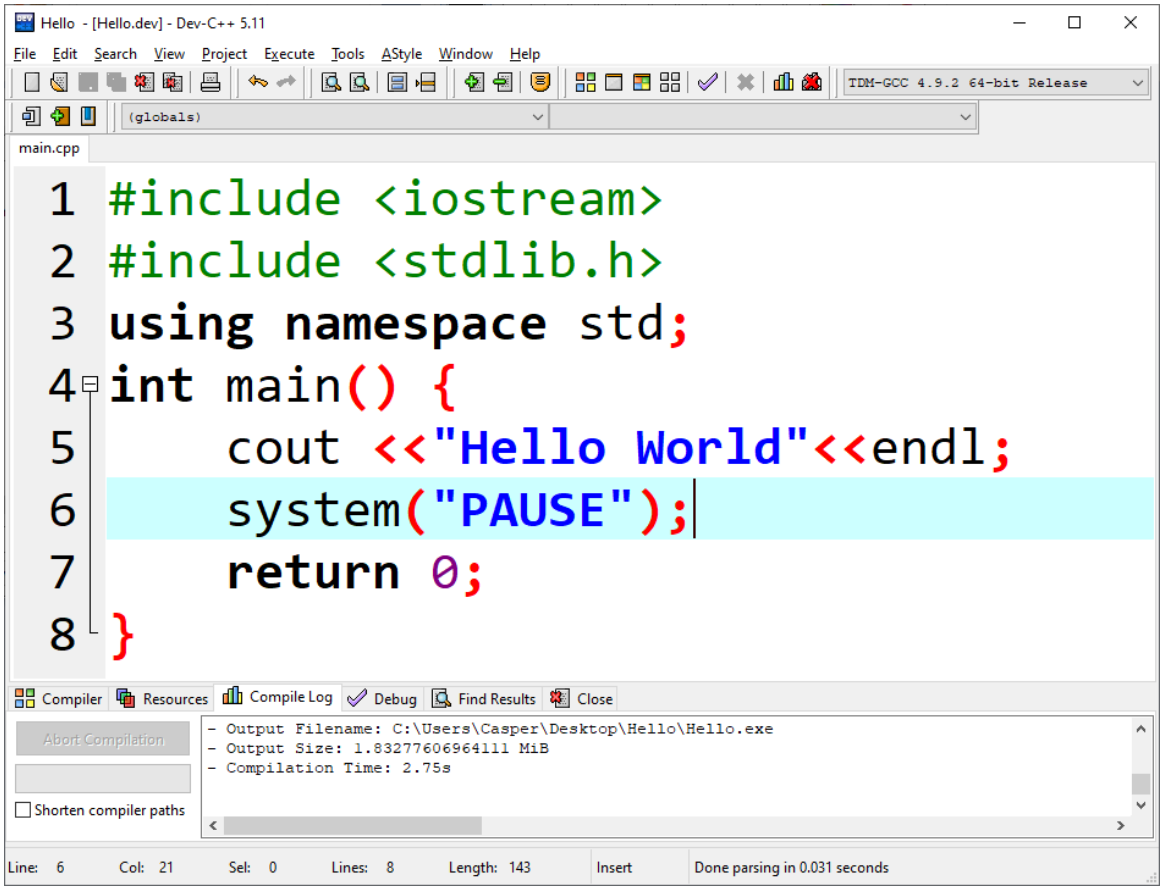



Add following code in **main.cpp** file

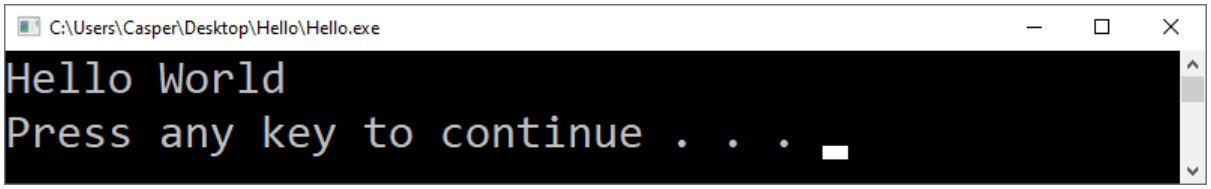
*Program 1.1: main.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
      cout <<"Hello World"<<endl;
      system("PAUSE");
      return 0;
}
```

**Save** the file into **Hello** folder with name **main.cpp** by pressing **Ctrl + S** buttons from keyboard.




Now compile and run the code using **Execute** > **Compile & Run** menu. You can also press **F11** key to do the same. After successful compilation the program will run and it will show following output on screen.




Congratulation new you have successfully compiled and run your program in C++.

## Task 1: Write your name and college name on screen.

## Here is the sample Output

My Name is ( Your Name ) and I did F.Sc. from ( College Name and City Name).

## Solution of Task 1:

# Lab 2 : Escape Sequence.

## Objective:

The objective of the lab is to understand how escape sequence are used in C++ programming.

## Escape Sequence:

In C++ programming, we can use some of the special characters with different behavior. These are called ***escape sequence***. The name reflects the fact that the backslash causes an "escape" from the normal way characters are interpreted. In this case the **\t** is interpreted not as the character **'t'** but as the **tab** character. A tab causes printing to continue at the next tab stop.

```
cout << "\"Run, \tForrest, run,\" she said.";
```

This translates to **"Run, Forrest, run," she said.**

The following table shows escape sequence characters with their ASCII values

| Escape Sequence | ASCII Value | Character | Purpose |
|---|---|---|---|
| \0 | 000 | Null | Null character. Marks the end of string |
| \a | 007 | Bell | A beep sound is generated |
| \b | 008 | Back Space | To erases one character from left side of cursor position |
| \t | 009 | Horizontal Tab | To shift cursor couple of spaces to the right |
| \n | 010 | New Line | To shift the cursor to the new line |
| \r | 013 | Carriage Return | To move cursor to the beginning of current line |
| \" | 034 | Quotation Mark ( " ) | To display double-quote character |
| \' | 039 | Apostrophe ( ' ) | To display single-quote character |
| \? | 063 | Question Mark ( ? ) | To display question mark character |
| \\ | 092 | Backslash ( \ ) | To display backslash character |

*Table 2.1: Escape Sequence Characters*

When the backslash character appears within quotes it is called the ***escape character***. If another character follows it immediately without a space, it is called an ***escape sequence***. The table above shows some of the escape sequences and the results that they produce.

```
cout << "This is my first program in C++\n";
```

As a result of the statement above, the string **This is my first program in C++** will be displayed on the screen and the cursor will move down to the beginning of the next line. The movement of the cursor to the next line is dictated by the ***escape sequence*** **\n**. It has the same effect as

```
cout << "This is my first program in C++" << endl;
```

**OR**

```
cout << "This is my first program in C++";
cout << endl;
```

Similarly, to move the cursor a certain number of spaces forward, without actually printing anything on the screen, we use the tab or **\t** escape sequence. Notice how the output will change if we write:

```
cout << "This\t is\t my\t first\t program\t in\t C++\n";
```

Similarly,

- **\r** takes the cursor to the beginning of the current line.
- **\b** is for backspace which will take you one space backwards. If **\b** is used immediately after **\t**, it has the effect of moving one whole tab backwards – just like in WinWord.

In addition to this, if you want to take input from the user in the first task. You wil use following code.

```
int variablename;
cout << "Enter a number: ";
cin  >> variablename;
```

Where 'variablename' is the name of an integer variable which has been defined earlier. These three statements will allow the user to enter any signed integer which will be stored in 'variablename'. Similarly,

```
char grade = 'B';          // the value assigned to a character is always written
// inside single quotes. assigned using ''.
cout << "Enter course grade: ";
cin >> grade;
```

Note: a variable of type **char** can only store one **character** at a time. Therefore, in the above case writing **grade = 'B+';** would have be illegal.

Write programs that display the output shown in the following diagrams.

Task 1:

Make a boat shape on screen

```
                    --
                  /||
                 / ||
                /  ||
               /   ||
              ----||
                  ||
                  ||
_________________||_____________
\                               /
 \                             /
  \___________________________/
Press any key to continue . . .
```

Task 2:

Show Following output on screen. Write your name inside a Box. Read ASCII Table to see characters needed to draw a double lined box.

```
╔═══════════════╗
║  Your Name     ╒
╚═══════════════╝

Press any key to continue . . .
```

## Solution of Task 1:

## Solution of Task 2:

# Lab 3 : User Input and Output.

## Objective:

The objective of the lab is to understand how to use cin for user input and cout for user output, in C++.

## Variables and constants:

The ability to change or not change the data stored can be third attribute of these memory locations. Components of memory in which data values stored can change during the execution of the program are called variables. These usually should not be defined in the header or global section of the program. In our sample program, radius is defined in the, main function. Components of the memory in which the data values stored are initialized once and never changed during the execution of the program are called constants.

## Identifiers in C++

Identifiers are used to name variables, constants and many other components of a program. They consist exclusively of letters, digits and the underscore _ character. They cannot begin with a digit and cannot duplicate reserved words used in C++ such as int or if. All characters in C++ are case sensitive; thus memory locations called simple, Simple, and SIMPLE are three distinct locations. It has become standard practice among programmers to make constants all uppercase and variables predominantly lowercase characters. The statement const double PI = 3.14; in our sample program is contained in the global section. It defines a memory location called PI to be a constant holding a double (a data type discussed shortly) value equal to 3.14 which will not change during the execution of the program. The statement float radius; in the sample program is contained in the main section. It defines a variable memory location called radius that holds a floating point data type (type discussed shortly) which can be changed during the execution of the program. Both of these statements are called definitions. They reserve by name enough memory to hold the data type specified. Variables, like constants, can be given an initial value when they are defined, but that value is not permanent and can be altered. For example:

```
int count = 7;
```

## Data Types

As noted earlier, computer memory is composed of locations identified by a data type and a name (like the room number of a hotel mailbox). The data type indicates what kind of data can be stored, thus setting the size of that location.

## Integer Data Type

Integers are real numbers that do not contain any fractional component. C++ has three data types that are integers: **short**, **int** and **long**. The difference is strictly in the amount of memory (bytes) they reserve: **short** reserving the least and **long** reserving the most. Larger integers may need the **long** data type.

The following three statements define integer variables in C++:

```
short count; int sum; long total;
```

## Floating Point Data Type

In computer science 3 = 3.0 is not a true statement. The number on the left is an integer and the number on the right is a real, or floating point, number (a number that has a fractional component). Although mathematically the two are equal, the computer stores them as different data types. C++ uses both float and double to indicate floating point numbers, with double using more memory than float. The following two statements define floating point variables in C++.

```
float average; double national_debt;
```

## Character Data Type

Character data includes the letters of the alphabet (upper and lower cases), the digits 0–9 and special characters such as ! ? . , *. All these symbols combined are called alphanumeric. Each character data is enclosed with single quotes to distinguish it from other data types. Thus '8' is different than 8. The first is a character while the second is an integer. The following statement defines a C++ character variable initialized to 'a'.

```
char letter = 'a';
```

## Boolean Data Type

The Boolean data type, named after the mathematician George Boole, allows only two values: true or false, which are internally represented as 0 and non-zero, respectively. The following statement defines a Boolean variable initialized to false.

```
bool found = false;
```

## String Type

A variable defined to be character data can store only one character in its memory location, which is not very useful for storing names. The string class has become part of standard C++ and, although not a primitive type defined by the language, it can be used as a type for storing several characters in a

memory location. We must “include” the string library (**#include <string>**) in the program header. The following statement defines a string initialized to “Daniel”:

```
string name = "Daniel";
```

Note that a string is enclosed in double (not single) quotes. Thus the string "a" is not the same as the character 'a'.

## Assignment Operator

The = symbol in C++ is called the assignment operator and is read “is assigned the value of.” It assigns the variable on its left the value on its right. Although this symbol looks like an equal sign, do not confuse it with equality. The left hand side must always be a variable. For example, **`count = 8;`** is a legitimate statement in C++, however 8 = count; generates a syntax error.

*Program 3.1: radius.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
const double PI  = 3.14;
int main() {
   float radius;
   radius = 4.0;
   cout << "PI = " << PI ;
   cout << "Radius = " << radius ;
   cout << "Circumference = " << 2 * PI * radius ;
   system("PAUSE");
   return 0;
}
```

We can express floating-point numbers in scientific notation. Since most programming editors do not provide superscripting and special symbols like ×, C++ slightly alters the normal scientific notation. The number **$6.022\times10^{23}$** is written **6.022e23**. The number to the left of the e (we can use capital **E** as well) is the **mantissa**, and the number to the right of the **e** is the **exponent** of 10. As another example **$5.1 \times 10^{-4}$** is expressed in C++ as **5.1e-4**.

*Program 3.2: scientific.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   double avogadros_number = 6.022e23, c = 2.998e8;
   cout << "Avogadro's number = " << avogadros_number ;
   cout << "Speed of light = " << c ;
   system("PAUSE");
   return 0;
}
```

*Program 3.3: charactter.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   char ch1, ch1 = 'A'; cout << ch1;
   system("PAUSE");
   return 0;
}
```

*Program 3.4: integer.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int count; int total, a = 10; count = 3 + 4;
   total = a + count;
   cout << "The value of total is " << total;
   system("PAUSE"); return 0;
}
```

## Using Cin

In most program environments, the standard input by default is the keyboard, and the C++ stream object defined to access it is cin. For formatted input operations, cin is used together with the extraction operator, which is written as >> (i.e., two "greater than" signs). This operator is then followed by the variable where the extracted data is stored. For example:

```
int age;  cin >> age;
```

The extraction operation on **cin** uses the type of the variable after the **>>** operator to determine how it interprets the characters read from the input; if it is an integer, the format expected is a series of digits, if a string a sequence of characters, etc.

*Program 3.5: cin.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int i;
   cout << "Please enter an integer value: ";
   cin >> i;
   cout << "The value you entered is " << i;
   system("PAUSE");
   return 0;
}
```

## TASK 1:

Assuming there are 7.48052 gallons in one cubic feet, write a program that asks to enter a number of gallons, and displays the equivalent in cubic feet.

The Format of output should be like this:

```
Enter Gallons: 10
10 Gallons = 1.3368 Cubic Feet.
```

## TASK 2:

Write a program that takes following two inputs from user and display that input, perform the all the arithmetic operations and display it on the console window again.

The Format of output should be like this:

```
Enter First  Number: -5
Enter Second Number: 3
-5 + 3 = -2
-5 – 3 = -8
-5 * 3 = -15
-5 / 3 = -1.6667
-5 % 3 = -2
```

## Solution of Task 1:

## Solution of Task 2:

# Lab 4 : Formatted Output.

## THE SETW( ) MANIPULATOR:

The **setw()** manipulator causes the number (or string) that follows it in the stream to be printed within a field n character wide, where n is the argument to setw().

- ✓ setw() is library function in C++.
- ✓ setw() is declared inside #include<iomanip>
- ✓ setw() will set field width.
- ✓ setw() sets the number of characters to be used as the field width for the next insertion operation.

The following program shows Lahore 10 characters wide.

*Program 4.1: setw1.cpp*

```
#include <iostream>
#include <iomanip>
#include <stdlib.h>
using namespace std;
int main() {
      cout <<setw(10)<<"Lahore"<<endl;
      system("PAUSE");
      return 0;
}
```

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | L | a | h | o | r | e |

Now Length of String Lahore is 6. We have set field width 10 so it will utilize 6 fields. Remaining 4 fields are kept blank.

*Program 4.2: setw2.cpp*

```
#include <iostream>
#include <iomanip>
#include <stdlib.h>
using namespace std;
int main() {
   cout << "HELLO" << setw(7) << "World" ;
```

```
    system("PAUSE");
    return 0;
}
```

In this program we use **cout** for display data on screen and it Shows HELLO World. There are 5 letters in **"World"** so **setw(7)** puts two spaces before showing **world** on screen. so the output is **HELLO<space><space>World**

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| H | E | L | L | O | | | W | O | R | L | D |

## Relational Operators:

In C++ Programming, the values stored in two variables can be compared using following operators and relation between them can be determined. Various C++ relational operators available are:

| Operator | Meaning |
|---|---|
| > | Greater than |
| < | Less than |
| == | Equal to |
| != | Not equal to |
| >= | Greater than or equal to |
| <= | Less than or equal to |

For example, **num1 > num2** does not assert that num1 is greater than num2. It actually tests to see if this is true. So relational expressions are **boolean**. Their value must be either **true** or **false**.

*Program 4.3: relational1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
    int age;
    cout<< "Enter the age"; cin >> age;
    cout <<"age >  10" << (age > 10) << endl;
    cout <<"age <  10" << (age < 10) << endl;
    cout <<"age == 10" << (age == 10) << endl;
    system("PAUSE");
    return 0;
}
```

*Program 4.4: relational2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int a, b;
   cout<<"Enter value a" << endl; cin >> a;
   cout<<"enter value b" << endl; cin >> b;
   c = a + b;
   cout<<"the output c = "<< c << endl;
   d = a - b;
   cout<<"the output d = "<< d << endl;
   cout<<" c <  d = " << (c <  d) << endl;
   cout<<" c >  d = " << (c >  d) << endl;
   cout<<" c == d = " << (c == d) << endl;
   cout<<" c <= d = " << (c <= d) << endl;
   cout<<" c >  d = " << (c >= d) << endl;
   cout<<" c != d = " << (c != d) << endl;
   system("PAUSE");
   return 0;
}
```

## TASK 1:

Write a program that generates the following output:

| | | |
|---|---|---|
| 1990 | 135 | 125 |
| 991 | 7290 | 2 |
| 1992 | 11399 | 111 |
| 93 | 16200 | 98 |

## TASK 2:

Write a program that follows the sequence use the appropriate date type that could deal with fractional inputs:

```
Enter temperature in Fahrenheit: 100
100 Fahrenheit = 37.7778 Celsius
Enter temperature Celsius: 37.5
73.5 Celsius = 99.5 Fahrenheit
```

**Hint:** `F = 32 + C * 5/9`

## Solution of Task 1:

## Solution of Task 2:

# Lab 5 : Bitwise Operators.

A central processing unit (CPU), also called a processor, is the electronic circuitry within a computer that carries out the instructions of a computer program by performing the basic arithmetic, logic, controlling, and input/output (I/O) operations specified by the instructions.

Main component of a CPU is the arithmetic logic unit (ALU) that performs arithmetic and logic operations. In C++ we can use bitwise operators to perform logic operations on our data. Bitwise Operators in C++ are

- Bitwise AND ( & )
- Bitwise OR ( | )
- Bitwise Exclusive OR ( ^ )
- Bitwise NOT ( ~ )
- Arithmetic Left Shift ( << )
- Arithmetic Right Shift ( >> )

Following are the bitwise operators and their truth table.

| A | B | A OR B<br>A \| B | A AND B<br>A & B | A XOR B<br>A ^ B | NOT A<br>~A | NOT B<br>~B |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

*Program 5.1: relational.cpp*

```
#include <iostream>   // for cin cout
#include <stdlib.h>   // for system()
using namespace std;  // for cin cout endl
int main(void) {
   short var1 = 0x6735 & 0x3C0F;      // bitwise AND
   short var2 = 0x6735 | 0x3C0F;      // bitwise OR
   short var3 = 0x6735 ^ 0x3C0F;      // bitwise XOR
   short var4 = ~ 0x6735;             // bitwise NOT
   cout << setw(4)<< hex << var1 <<endl
```

```
        << setw(4)<< hex << var2 <<endl
        << setw(4)<< hex << var3 <<endl
        << setw(4)<< hex << var4 <<endl;
   system("PAUSE");
   return 0;
}
```

Following is the explanation of output

```
       0x6735 = 0110 0111 0011 0101
       0x3C0F = 0011 1100 0000 1111; & bitwise AND
------------------------------------
var1 = 0x2405 = 0010 0100 0000 0101
------------------------------------
       0x6735 = 0110 0111 0011 0101
       0x3C0F = 0011 1100 0000 1111; | bitwise OR
------------------------------------
var2 = 0x7F3F = 0111 1111 0011 1111
------------------------------------
       0x6735 = 0110 0111 0011 0101
       0x3C0F = 0011 1100 0000 1111; ^ bitwise XOR
------------------------------------
var3 = 0x5B3A = 0101 1011 0011 1010
------------------------------------
       0x6735 = 0110 0111 0011 0101
var4 = 0x98CA = 1001 1000 1100 1010 ~ // bitwise NOT
```

## Shift Operators:

Shift operations are also a kind of logic operations.

In C or C++ Programming, we can use

- << operator for Arithmetic Left Shift.
- >> operator for Arithmetic Right Shift.

*Program 5.2: shift.cpp*

```
#include <iostream>   // for cin cout
#include <stdlib.h>   // for system()
using namespace std;  // for cin cout endl
int main(void) {
   short n;
   cout << "Enter a number: "; cin  >> n;
   cout << n << " << 1 = " << (n << 1) << endl;
   cout << n << " << 2 = " << (n << 2) << endl;
   cout << n << " << 3 = " << (n << 3) << endl;
   cout << n << " << 4 = " << (n << 4) << endl << endl;
   cout << n << " >> 1 = " << (n >> 1) << endl;
   cout << n << " >> 2 = " << (n >> 2) << endl;
   cout << n << " >> 3 = " << (n >> 3) << endl;
   cout << n << " >> 4 = " << (n >> 4) << endl;
   system("PAUSE");
   return 0;
}
```

Following is the explanation of output

```
+7       = 0000 0000 0000 0111
-7       = 1111 1111 1111 1001
-7 << 1  = 1111 1111 1111 0010   = -14
-7 << 2  = 1111 1111 1110 0100   = -28
-7 << 3  = 1111 1111 1100 1000   = -56
-7 << 4  = 1111 1111 1001 0000   = -112
----------------------------------------
-7       = 1111 1111 1111 1001
-7 >> 1  = 1111 1111 1111 1100   = -4
-7 >> 2  = 1111 1111 1111 1110   = -2
-7 >> 3  = 1111 1111 1111 1111   = -1
-7 >> 4  = 1111 1111 1111 1111   = -1
----------------------------------------
+7       = 0000 0000 0000 0111
```

```
+7 >> 1  = 0000 0000 0000 0011   = +3
```

## TASK 1:

Write a program that takes two integer numbers from a user and shows the answers after performing bitwise AND, OR and XOR on these two Numbers:

Your output should be like this. Solve these on a paper to verify the answers.

```
Enter 1st Number: 100
Enter 2nd Number: 200
100 & 200 = 64
100 | 200 = 236
100 ^ 200 = 172
Press any key to continue . . .
```

## TASK 2:

Write a program that takes a Number from user and performs arithmetic shift left and shift right operations on these numbers:

Your output should be like this. Solve these on a paper to verify the answers.

```
Enter a Number: -50
-50 << 0 = -50
-50 << 2 = -200
-50 << 4 = -800
-50 << 6 = -3200
-50 << 8 = -12800
-50 << 10 = -51200

-50 >> 0 = -50
-50 >> 2 = -13
-50 >> 4 = -4
-50 >> 6 = -1
-50 >> 8 = -1
-50 >> 10 = -1
Press any key to continue . . .
```

## Solution of Task 1:

## Solution of Task 2:

# Lab 6 : Logic Operators.

Following are the logical operators in C++.

| Operator | Description | Example Code |
|---|---|---|
| && | Logical AND operator: If both the operands are non-zero, then condition becomes true. | A && B |
| \|\| | Logical OR Operator: If any of the two operands is non-zero, then condition becomes true. | A \|\| B |
| ! | Logical NOT Operator: Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make false. | ! A |

## DECISION:

### The if Statement

Sometimes we may only want a portion of code executed under certain conditions. To do so, we use conditional statements. An **if** statement is one kind of conditional statement. Consider the following program:

*Program 6.1: if.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   float average;
   cout << "Input your average" << endl;
   cin >> average;
   if (average >= 60) // note the use of a relational operator
      cout << "You Pass" << endl;
   if (average < 60)
      cout << "You Fail" << endl;
   system("PAUSE");
   return 0;
}
```

This program prints "You Pass" if a student's average is 60 or higher and prints "You Fail" otherwise.

Note that it is not possible for this program to print out both “You Pass” and “You Fail”. Only one of the if statements will be executed. Later we will see a way to write this program without using 2 if statements.

A more elegant approach would be to use the if/else statement as follows:

The if/else if Statement The if/else statement works well if there are only two possible paths to follow.

*Program 6.2: ifelse.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   float bonus;
   cout << "Please input the amount of your yearly bonus"
        << endl;
   cin  >> bonus;
   if (bonus < 1000)
      cout << "Another vacation eating hot dogs on the lawn"
           << endl;
   else
      if (bonus > 10000)
         cout << "Off to Disney World!" << endl;
      else
         if (bonus == 10000)
            cout << "Lets go to Hawaii!" << endl;
   system("PAUSE");
   return 0;
}
```

## Nested if Statements:

Often programmers use an if statement within another if statement. For example, suppose a software engineering company wants to screen applicants first for experienced programmers and second for C++ programmers specifically we will write following code.

*Program 6.3: nested_if.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   char programmer, cPlusPlus;
   cout << "Before we consider your application,"
            "answer the following" << endl;
   cout << " yes ( enter Y ) or no ( enter N )" << endl;
   cout << "Are you a computer programmer?" << endl;
   cin >> programmer;
   if (programmer == 'Y') {
      cout << "Do you program in C++?" << endl;
      cin >> cPlusPlus;
      if (cPlusPlus == 'Y')
         cout << " You look like a promising candidate for"
                 " employment" << endl;
      else
         if (cPlusPlus == 'N')
            cout << " You need to learn C++ before further"
                    " consideration" << endl;
         else
            cout << " You must enter Y or N" << endl;
   }
   else
      if (programmer == 'N')
         cout << " You are not currently qualified for"
                 " employment" << endl;
      else
         cout << " You must enter Y or N" << endl;
   system("PAUSE");
   return 0;
}
```

*Program 6.4: compound.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int a = 5; int b = 20; int c ;
   if ( a && b ) {
      cout << "Line 1 - Condition is true"<< endl ;
   }
   if ( a || b ) {
      cout << "Line 2 - Condition is true"<< endl ;
   }
   a = 0; b = 10;  // Let's change the values of a and b
   if ( a && b ) {
      cout << "Line 3 - Condition is true"<< endl ;
   }
   else {
      cout << "Line 4 - Condition is not true"<< endl ;
   }
   if ( !(a && b) ) {
      cout << "Line 5 - Condition is true"<< endl ;
   }
   system("PAUSE");
   return 0;
}
```

This will be the output after executing above code

```
Line 1 - Condition is true
Line 2 - Condition is true
Line 4 - Condition is not true
Line 5 - Condition is true
Press any key to continue . . .
```

## TASK 1:

Write a program to ask the user to enter age

- if the age is less than 0 or above 160 display "You're lying - you CANNOT be that age.
- if the age is in between 35 to 80 display “You can save money on your car insurance”
- if the age is in between 0 to 34 or 81 to 160 display "Sorry, we don't have any deals for you today!"

## TASK 2:

Write a program to ask the student "What year student are you? Give them the choice to enter value “1 2 3 4”. Then ask the student to enter GPA . Identify which students will graduate and which will not graduate according to these conditions.

- if gpa is greater than or equal to 2.5 and year greater than or equal to '4' he/she will graduate
- if year is less than '4' or gpa is less than 2.5 he/she will not graduate

## Solution of Task 1:

## Solution of Task 2:

# Lab 7 : Decisions and Loops.

We have already seen how **if** statements can affect the branching of a program during execution. Another way to do this is using the **switch** statement. It is also a conditional statement. The switch statement uses the value of an integer expression to determine which group of statements to branch through. The sample program below illustrates the syntax.

*Program 7.1: switch1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   char grade;
   cout << "What grade did you earn in Programming I?" << endl;
   cin >> grade;
   switch( grade ) { // This is where the switch statement begins
      case 'A': cout <<"an A - excellent work!" << endl; break;
      case 'B': cout <<"you got a B - good job" << endl; break;
      case 'C': cout <<"earning a C is satisfactory"
                     << endl; break;
      case 'D': cout << "while D is passing, there is a problem"
                     << endl; break;
      case 'F': cout << "you failed - better luck next time"
                     << endl; break;
      default:  cout << "You did not enter an A, B, C, D, or F"
                     << endl;
   }
   system("PAUSE");
   return 0;
}
```

Note the use of the curly braces that enclose the cases and the use of **break;** after each case. Also, consider the variable grade. It is defined as a character data type and the **case** statements have character arguments such as 'B'. This seems to contradict what we said above, namely that the switch statement uses the value of integer expressions to determine branching. However, this apparent

contradiction is resolved by the compiler automatically converting character data into the integer data type. Finally, notice the role of the **default** statement. The default branch is followed if none of the case expressions match the given **switch** expression.

*Program 7.2: switch2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int value;
   cout << "Please enter an integer in the range 0...5: ";
   cin >> value;
   switch (value) {
      case 0: cout << "zero" ; break;
      case 1: cout << "one"  ; break;
      case 2: cout << "two"  ; break;
      case 3: cout << "three"; break;
      case 4: cout << "four" ; break;
      case 5: cout << "five" ; break;
      default: if (value < 0)
                  cout << "Too small";
               else
                  cout << "Too large"; break;
   }
   cout << endl;
   system("PAUSE");
   return 0;
```

## THE CONDITIONAL OPERATOR

As purely a syntactical convenience, C++ provides an alternative to the if/else construct called the conditional operator. It has limited application but is convenient nonetheless. The following code fragment assigns one of two things to x:

- The result of y/z, if z is nonzero, or
- Zero, if z is zero;

We wish to avoid the run-time error of division by zero.

```
// Assign a value to x:
if (z != 0)
   x = y/z; // Division is possible
else
   x = 0; // Assign a default value instead
```

This code has two assignment statements, but only one is executed at any given time. The conditional operator makes for a more compact statement:

```
// Assign a value to x:
x = (z! = 0) ? y/z: 0;
```

The general form of a conditional expression is

```
(Condition)     ?     Expression 1     :     Expression 2;
```

- Condition is a normal Boolean expression that might appear in an **if** statement. Parentheses around the condition are not required but should be used to improve the readability.
- Expression 1 is the overall value of the conditional expression if condition is **true**.
- Expression 2 is the overall value of the conditional expression if condition is **false**.

## LOOPS

- Loops cause a section of your program to be repeated a certain number of times.
- The repetition continues while a condition is **true**.
- When the condition becomes **false**, the loop **ends** and control passes to the statements following the loop.
- There are three kinds of loops in C++:
  - **for** loop
  - **while** loop
  - **do...while** loop

### WHILE LOOP

Often in programming one needs a statement or block of statements to repeat during execution. This can be accomplished using a loop. A loop is a control structure that causes repetition of code within a program. C++ has three types of loops. The first we will consider is the **while** loop. The following is the syntax:

```
while (expression) {
      statement_1;
      statement_2;
      statement_n;
}
```

If there is only one statement, then the curly braces can be omitted. When a **while** loop is encountered during execution, the expression is tested to see if it is **true** or **false**. The block of statements is repeated as long as the expression is **true**. Consider the following program

*Program 7.3: while1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int num = 5; int numFac = 1;
   while (num > 0) {
      numFac = numFac * num;
      num --; // note the use of the decrement operator
   }
   cout << " 5! = " << numFac << endl;
   system("PAUSE");
   return 0;
}
```

This program computes 5! = 5 * 4 * 3 * 2 * 1 and then prints the result to the screen. Note how the **while** loop controls the execution. Since **num = 5** when the **while** loop is first encountered, the block of statements in the body of the loop is executed at least once. In fact, the block is executed 5 times because of the decrement operator which forces the value of **num** to decrease by one every time the block is executed. During the fifth iteration of the loop **num** becomes 0, so the next time the expression is tested **num > 0** is **false** and the loop is exited. Then the **cout** statement is executed.

What do you think will happen if we eliminated the decrement operator **num--** in the above code? The value of **num** is always 5. This means that the expression **num > 0** is always true! If we try to execute the modified program, the result is an **infinite loop**, i.e., a block of code that will repeat forever.

*Program 7.4: while2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   char letter = 'a';
   while (letter != 'x') {
      cout << "Please enter a letter" << endl;
      cin >> letter;
      cout << "The letter your entered is " << letter << endl;
   }
   system("PAUSE");
   return 0;
}
```

## SUMMARY

The **if/else** and **while** statements are sufficient to implement any algorithms that involve conditional execution and looping. The **break** and **continue** statements are convenient but are not necessary. C++ provides some additional conditional and iterative statements that are more convenient to use in some circumstances. These additional statements include

- **switch**: an alternative to some multi-way **if/else** statements
- The conditional operator: an expression that exhibits the behavior of an **if/else** statement
- **do...while**: a loop that checks its condition after its body is executed
- **for**: a loop convenient for counting.

## TASK 1:

Write a program in which the user can input data (integer type) and then a **while** loop tests to see if the value(s) is valid. The loop is skipped for all valid input but for invalid input (float type ) the loop is executed and prompts the user to enter new (valid) input. Input data should be the drink that user want to drink.

The beverages are:

1 – Coffee 2 – Tea 3 – Coke 4 – Orange juice

## TASK 2 :

Create the equivalent of a four-function calculator. The program should ask the user to enter a **number**, an **operator**, and **another number**. (Use floating point.) It should then carry out the specified arithmetical operation: **adding**, **subtracting**, **multiplying**, or **dividing** the two numbers.

- Use a switch statement to select the operation. Finally, display the result. When it finishes the calculation, the program should ask whether the user wants to do another calculation. The response can be 'y' or 'n'.
- Some sample interaction with the program might look like this:

```
Enter <first number> <operator> <second number>: 10 / 3
answer = 3.333333 Do another (y/n)? y
Enter <first number> <operator> <second number>: 12 + 100
answer = 112 Do another (y/n)? n
```

## Solution of Task 1:

## Solution of Task 2:

# Lab 8 : Do…While and For Loop.

A loop that checks its condition after its body is executed at least once would be more appropriate. The **do…while** statement is a bottom-checking loop.

- The reserved words **do** and **while** identify a **do…while** statement. The **do** and **while** keywords delimit the loop's body, but curly braces are still required if the body consists of more than one statement.
- The condition is associated with the **while** at the end of the loop. The condition is a **boolean** expression and must be enclosed within parentheses.
- The statement is exactly like the statement in the general form of the **while** loop .It can be a compound statement enclosed within curly braces.
- The body of a **do…while** statement, unlike the **while** statement, is guaranteed to execute at least once. The **do…while** loop is a convenience to the programmer and is not an essential programming construct. It is easy to transform any code that uses a **do…while** statement into code that behaves identically that uses a **while** statement instead.
- In practice, programmers use **while** loops much more frequently than **do…while** loops because more algorithms require top-checking loops than bottom-checking loops.

*Program 8.1: do_while1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int in_value;
   cout << "Please enter an integer in the range 0-10: ";
   do
      cin >> in_value;
   while (in_value < 0 || in_value > 10);
   cout << "Legal value entered was” << in_value << endl;
   system("PAUSE");
   return 0;
}
```

*Program 8.2: do_while2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   int a = 10;
   do {
      cout << "value of a: " << a << endl;
   } while( a < 20 );
   system("PAUSE");
   return 0;
}
```

## FOR LOOP:

It simply counts from one to five. Counting is a frequent activity performed by computer programs. Certain program elements are required in order for any program to count:

- A variable must be used to keep track of the count; count is the aptly named counter variable.
- The counter variable must be given an initial value. In the case of the initial value is 1.
- The variable must be modified (usually incremented) as the program counts. The statement count++;
- A way must be provided to determine if the counting has completed. The condition of the while statement determines if the counting is complete or must continue. C++ provides a specialized loop that packages these four programming elements into one convenient statement. Called the **for** statement, its general form is

```
for ( initialization ; condition ; modification )
      statement
```

- The reserved word for identifies a for statement.
- The loop is controlled by a special variable called the loop variable.
- The header, contained in parentheses, contains three parts, each separated by semicolons:
    - Initialization. The initialization part assigns an initial value to the loop variable. The loop variable may be declared here as well; if it is declared here, then its scope is limited to the for statement. This means you may use that loop variable only within the loop. It also means you are free to reuse that variable's name outside the loop to

declare a different variable with the same name as the loop variable. The initialization part is performed one time.

- Condition. The condition part is a Boolean expression, just like the condition of a while statement. The condition is checked each time before the body is executed.
- Modification. The modification part generally changes the loop variable. The change should be suchthattheconditionwilleventuallybecomefalsesotheloopwillterminate. The modification is performed during each iteration after the body is executed. Notice that the last part (modification) is not following by a semicolon; semicolons are used strictly to separate the three parts.

- The statement is like the body of any other loop. It may be a compound statement within curly braces.
- Any for loop can be rewritten as a while loop. The general form of the for loop given above can be written equivalently as

```
Initialization
while (condition) {
      statement
      modification
}
```

*Program 8.3: for1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
   for ( int count = 1 ; count <= 5 ; count++)
      cout << count << endl;  // Display counter
   system("PAUSE");
   return 0;
}
```

With a **while** loop, the four counting components (variable declaration, initialization, condition, and modification can be scattered throughout the code. With a **for** loop, a programmer should be able to determine all the important information about the loop's control by looking at one statement.

*Program 8.4: for2.cpp*

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
  int size;
  // The number of rows and columns in the table
  cout << "Please enter the table size: ";
  cin >> size;
  // Print a size x size multiplication table
  // First, print heading
  cout << "     |";
  for (int column = 1; column <= size; column++)
    cout << setw(4) << column;
  // Print heading for this column.
  cout << endl; // Print line separator
  cout << " +----";
  for (int column = 1; column <= size; column++)
    cout << "----";
  // Print separator for this column.
  cout << endl;
  // Print table contents
  for (int row = 1; row <= size; row++)
  {
    cout << setw(4) << row << " |";
    // Print row label.
    for (int column = 1; column <= size; column++)
      cout << setw(4) << row * column;
    // Display product
    cout << endl; // Move cursor to next row
  }
}
```

This will be the output of program

```
Please enter the table size: 10
     |   1   2   3   4   5   6   7   8   9  10
 +-------------------------------------------
   1 |   1   2   3   4   5   6   7   8   9  10
   2 |   2   4   6   8  10  12  14  16  18  20
   3 |   3   6   9  12  15  18  21  24  27  30
   4 |   4   8  12  16  20  24  28  32  36  40
   5 |   5  10  15  20  25  30  35  40  45  50
   6 |   6  12  18  24  30  36  42  48  54  60
   7 |   7  14  21  28  35  42  49  56  63  70
   8 |   8  16  24  32  40  48  56  64  72  80
   9 |   9  18  27  36  45  54  63  72  81  90
  10 |  10  20  30  40  50  60  70  80  90 100
Press any key to continue . . .
```

## Task 1:

Write a program that prints the squares of the numbers from 1 to 20, using a two-column format using **for** loop.

```
 1^2 =   1
 2^2 =   4
 …
20^2 = 400
```

## Task 2:

Write a program that subtracts two numbers until and unless user enters first number equals to 1 using **do…while** loop ?

## Solution of Task 1:

## Solution of Task 2:

# Lab 9 : Functions.

A function groups a number of program statements into a unit and gives it a name. This unit can then be invoked from other parts of the program. A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function. Functions has three main parts

- Function prototype → declares the function
- Function Definition → defines what the function will do
- Function Call → asks the function to come and perform the task.

*Program 9.1: function1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int cube(int x);      // function declaration
int main(){           // tests the cube() function:
     int n = 1;
     while (n != 0){
          cout <<"Enter a Number: "; cin >> n;
          cout  << "\tcube(" << n << ") = "
               << cube(n) << endl;    // Calling a function
     }    // end of while loop
     system("PAUSE"); return 0;
}//end of main

int cube( int x ){    // function definition
     return x*x*x;    // returns cube of x:
}                     // { function body }
```

In this program

- Each integer read is passed to the **`cube()`** function by the call **`cube(n)`**.
- The value returned by the function replaces the expression **`cube(n)`** and then is passed to the output object **`cout`**.
- The **`main()`** function passes the value 5 to the **`cube()`** function, and the **`cube()`** function returns the value 125 to the **`main()`** function.

- The argument **n** is **passed by value** to the formal parameter **x**. This simply means that **x** is assigned the value of **n** when the function is called.

## Passing by Reference

*Program 9.2: function2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
void compute( double&, double&, const double& );
int main(){ // tests the computeCircle() function:
    double r,a,c; cout << "Enter radius: "; cin >> r;
    compute(a, c, r);
    cout << "Area = " <<a<<", Circumference = " <<c<< endl;
    system("PAUSE"); return 0;
} // end of main

// this function returns the area and circumference of a circle
void compute(double& area, double& circumference,
                    const double& r){
    const double PI = 3.141592653589793;
    area = PI*r*r;
    circumference = 2*PI*r;
}
```

## TASK 1:

Ask the user to enter the integer value, once the value is entered, it should be shown whether it is an odd number or even number. Make two function one is named as **`void odd (int)`** other should be named as **`void even(int)`**. The program must execute until user enters **0** value.

## TASK 2:

Write a function called **`zero_smaller(int &, int &)`** that is passed two **int** arguments by reference and then sets the smaller of the two numbers to **0**. Write a **`main()`** program to exercise this function.

## Solution of Task 1:

## Solution of Task 2:

# Lab 10 : Inline and Recursive Functions.

## Inline Function

In C++, inline function is powerful concept that is commonly used with classes. If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.

- Any change to an inline function could require all clients of the function to be recompiled because compiler would need to replace all the code once again otherwise it will continue with old functionality.
- To inline a function, place the keyword **inline** before the function name and define the function before any calls are made to the function. The compiler can ignore the inline qualifier in case defined function is more than a line.

The following example makes use of inline function to return max of two numbers:

*Program 10.1: inline.cpp*

```
#include <iostream>
using namespace std;

inline int Max(int x, int y) {
  return (x > y) ? x : y;
}
int main()
{
  cout << "Max (20,10)   : " << Max( 20,  10) << endl;
  cout << "Max (0,200)   : " << Max(  0, 200) << endl;
  cout << "Max (100,1010): " << Max(100,1010) << endl;
  system("PAUSE");
  return 0;
}
```

## Recursion

The factorial function is widely used in combinatorial analysis (counting theory in mathematics), probability theory, and statistics. The factorial of **n** is often expressed as **n!**. Factorial is defined for non-negative integers as **n! = n·(n–1)·(n–2)·(n–3)···2·1**

and **0!** is defined to be **1**. Thus **6! = 6·5·4·3·2·1 = 720**. Mathematicians precisely define factorial in this way:

**If n = 0 then n! = 1**

**Otherwise n! = n·(n–1)!**

The following program shows its implementation in C++

*Program 10.2: factorial.cpp*

```
#include <iostream>
using namespace std;

int factorial(int n) {
  if (n == 0)
    return 1;
  return n * factorial(n - 1);
}

int main() {
  cout << " 0! = " << factorial(0) << endl;
  cout << " 1! = " << factorial(1) << endl;
  cout << " 6! = " << factorial(6) << endl;
  cout << "10! = " << factorial(10) << endl;
  system("PAUSE");
  return 0;
}
```

Observe that the factorial function in this program uses no loop to compute its result. The factorial function simply calls itself. The call factorial(6) is computed as follows:

factorial(6) = 6 * factorial(5)

= 6 * 5 * factorial(4)

= 6 * 5 * 4 * factorial(3)

= 6 * 5 * 4 * 3 * factorial(2)

= 6 * 5 * 4 * 3 * 2 * factorial(1)

= 6 * 5 * 4 * 3 * 2 * 1 * factorial(0)

= 6 * 5 * 4 * 3 * 2 * 1 * 1

= 6 * 5 * 4 * 3 * 2 * 1

= 6 * 5 * 4 * 3 * 2

= 6 * 5 * 4 * 6

= 6 * 5 * 24

= 6 * 120

= 720

## TASK 1:

Calculate Sum of Natural numbers using Recursion:

## TASK 2:

Make a function **`bool Is_Digit(char n)`** which returns **true** if the character is between character 0 to 9 **else** it returns **false** using **inline** function.

## Solution of Task 1:

## Solution of Task 2:

# Lab 11 : Arrays.

## One-Dimensional Arrays

In everyday life we commonly group similar objects into units. We buy eggs by the carton. In computer languages we also need to **group** together data items of the **same type**. The most basic mechanism that accomplishes this in C++ is the **array**. Arrays can hold a few data items or tens of thousands. The data items grouped in an array can be simple types such as **int** or **float**, or they can be user-defined types such as **structures** and **objects**. An array groups items of the same type. The items in a in an array are accessed by an index number. Using an **index number** to specify an item allows easy access to a large number of items.

## Defining, Reading and Writing Array

*Program 11.1: array1.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
  int age[4], j; //array 'age' of 4 ints
  for (j = 0; j < 4; j++) { //get 4 ages
    cout << "Enter an age: ";
    cin >> age[j]; //access array element
  }
  for (j = 0; j < 4; j++) // display 4 ages
    cout << "age[" << j << "] = " << age[j] << endl;
  system("PAUSE");
  return 0;
}
```

In this program we defined an array of four integers and the name of array is age.

In the next program we ask user to enter 10 Numbers. Using for loop we store the numbers in an array. Again using for loop we sum all the numbers. Then we divide the sum by 10 to get an average.

*Program 11.2: array2.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main(){
      double avg, sum = 0 ;
      int i ;
      int marks[10] ;            /* array declaration */
      for ( i = 0 ; i <= 9 ; i++ ){
            cout << "\nEnter marks ";
            cin >> marks[i];      /* store data in array */
      }
      for ( i = 0 ; i <= 9 ; i++ )
            sum = sum + marks[i] ; /* read data from array*/
      avg = sum / 10 ;
      cout << "\n Average marks = " << avg <<endl;
      system("PAUSE");
      return 0;
}
```

## Passing 1 Dimensional Array to a Function.

When we pass an array to a function. The same original array is passed to the function. Function does not make its duplicate. When we call a function and pass it an array then function can change the data of this array. To duplicate an array, we have to write extra code inside array. In the In the Program 11.3 we have made a function named sum. The function takes two arguments. The first argument is an array and the second argument is size of an array. The function calculates the sum of all elements of an array then its returns the answer.

## Passing 2 Dimensional Array to a Function.

In the In the Program 11.4 we have made a function named display. The function takes three arguments. The first argument is an array and the second number of rows and the third argument is number of columns in an array.

*Program 11.4: array3.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int sum(int[],int); // function declaration

int main(){
      int a[] = { 11,33, 55,77 };
      int size = sizeof(a) / sizeof(int);
      cout << "sum(a,size) = " << sum(a,size) << endl;
      system("PAUSE"); return 0;
}
int sum(int a[],int n){    // function definition
      int sum=0;
      for (int i=0; i<n; i++)
            sum += a[i];
      return sum;
}
```

*Program 11.4: array3.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
void display(int n[ ][4], int row, int col);
int main() {
      int num[3][4] = {
                              {11, 12, 13, 14},
                              {21, 22, 23, 24},
                              {31, 32, 33, 34}
                        };
      display(num);
      return 0;
}
```

```
void display(int n[ ][4], int row, int col) {
      for ( int i = 0 ;  i < row ; i++) {
            for ( int j = 0; j < col; j++) {
                  cout << n[i][j] << " ";
            }
            cout << endl;
      }
}
```

TASK 1:

Make a two argument function called **`average (double arr, int sz)`**, Call this function in **`min()`** to calculated average of numbers stored in an array.

TASK 2:

Write and test the following function: **`double Standard_Deviation (double x[], int n);`**
The function returns the Standard Deviation of numbers stored in an array. The mathematical formula for calculation of standard deviation is

$$s = \sqrt{\frac{\sum_{i=0}^{n-1} (x_i - \bar{x})^2}{n-1}}$$

## Solution of Task 1:

## Solution of Task 2:

# Lab 12 : String

String is a collection of characters. There are two types of strings commonly used in C++ programming language:

- String that is an object of string class (The Standard C++ Library string class)
- C String is an array of type **char** terminated with a **null** character, that is, **'\0'**. The ASCII value of **null** character is 0.

## How to define a C String?

```
char str[] = "C++";
```

In the above code, **str** is a string and it holds 4 characters. Although, **"C++"** has 3 characters, the null character **'\0'** is added to the end of the string automatically.

Alternative ways of defining a string

```
char str[4] = "C++";
char str[ ] = {'C','+','+','\0'};
char str[4] = {'C','+','+','\0'};
```

Like arrays, it is not necessary to use all the space allocated for the string. For example:

```
char str[100] = "C++";
```

The following program displays your name and your friend's name entered by the you.

*Program 12.1: c_string.cpp displays a c string entered by the user.*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main( ){
      const int SIZE = 100;       char str[SIZE];
      cout << "Enter your name: ";
      cin.get(str,SIZE);          fflush(stdin);
      cout << "Assalam O Alikum " << str << endl;
      cout << "\nEnter your firend\'s name: ";
      cin.get(str,SIZE);          fflush(stdin);
      cout << "Where is your friend "<< str << endl;
      system("PAUSE"); return 0;
}
```

This will be the output of program

```
Enter your name: Rashid Farid
Assalam O Alikum Rashid Farid

Enter your firend's name: M. Bilal
Where is your friend M. Bilal
Press any key to continue . . .
```

To read the text containing blank space, **`cin.get(char* _Str, int _Count)`** function can be used. This function takes two arguments. The First argument is the name of the string (address of first element of string) or a character array and the second argument is the maximum size of that array.

**`fflush(stdin);`** is used to empty the standard input buffer.

Similarly, we can use **`fflush(stdout);`** to empty the standard output buffer.

## Passing a C String to a Function

A C String can be passed to a function in a similar way as an integer array is passed to a function.

*Program 12.2: function.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
void display(char s[]);
int main( ){
      char str[100];
      cout << "Enter a message: ";
      cin.get(str,100);
      display(str);
      system("PAUSE");
      return 0;
}
void display(char s[]) {
      cout << "Your message was " << s <<endl;
}
```

This will be the output of program

```
Enter a message: Hello World
Your message was Hello World
Press any key to continue . . .
```

## TASK 1:

Write a program to find the number of times a character “a” is appeared in a C Sting “We are shinning stars of Pakistan”.

## TASK 2:

Write program takes a text from the user and remove all the characters except alphabets.

## Solution of Task 1:

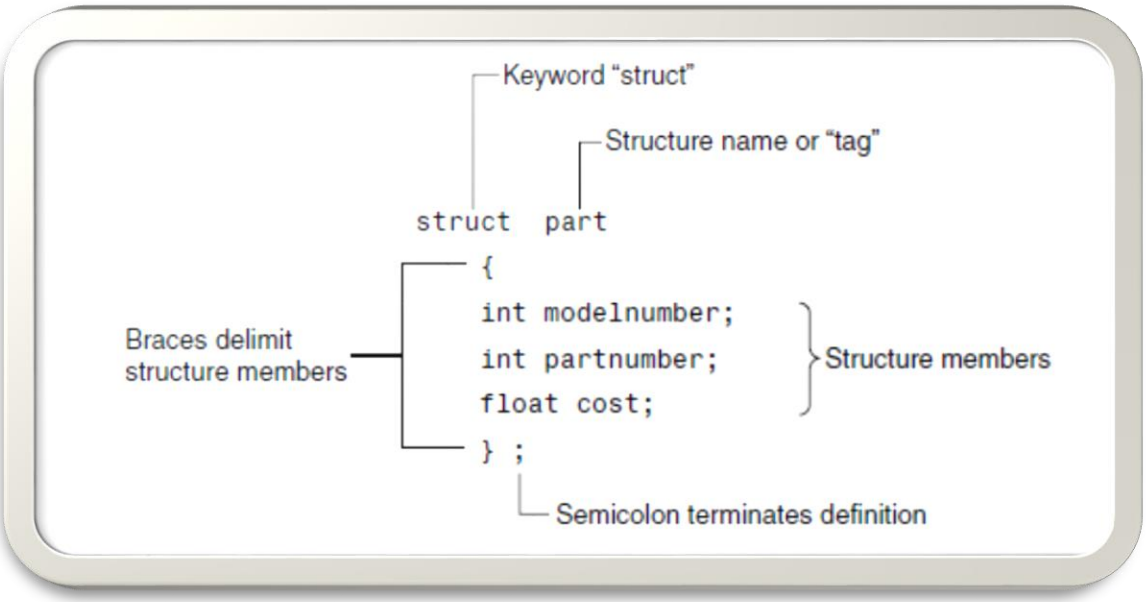
## Solution of Task 2:

# Lab 13 : Structure.

We have seen variables of simple data types, such as **float**, **char**, and **int**. Variables of such types represent one item of information: a height, an amount, a count, and so on. But just as groceries are organized into bags, employees into departments, and words into sentences, it's often convenient to organize simple variables into more complex entities.

The C++ construction called the **structure** is one way to do this. A **structure** is a collection of simple variables (can be of different types). The data items in a **structure** are called the **members** of the structure.

Let's start off with a structure that contains three variables: two integers and a floating-point number. This structure represents an item in a widget company's parts inventory. The structure is a kind of blueprint specifying what information is necessary for a single part. The company makes several kinds of widgets, so the widget **model number** is the first member of the structure. The **number of the part** itself is the next member, and the final member is the part's **cost**. The next program defines the **structure part**, defines a **structure variable** of that type called **part1**, assigns values to its members, and then displays these values.



*Program 13.1: structure.cpp*

```
// uses parts inventory to demonstrate structures
#include <iostream>
using namespace std;
struct part{                // declare a structure
      int modelnumber;      // ID number of widget
      int partnumber;       // ID number of widget part
      float cost;           // cost of part
};
int main(){
      part part1;           // define a structure variable
                            // give values to structure members
      part1.modelnumber = 6244;
      part1.partnumber  = 373;
      part1.cost        = 217.55F;
      // display structure members
      cout << "Model "      << part1.modelnumber;
      cout << ", part "     << part1.partnumber;
      cout << ", costs $"   << part1.cost << endl;
      system("PAUSE");
      return 0;
}
```

This will be the output of program

```
Model 6244, part 373, costs $217.55
Press any key to continue . . .
```

We can define a structure variable **part1** and initialize it using a single statement.

```
part part1 = { 6244, 373, 217.55F };
```

So **6244** will be copied into **modelnumber**, **373** will be copied into **partnumber** and **217.55** will be copied into **cost**. We can also copy one structure variable **part1** into other structure variable **part2** by using assignment operator = e.g.

```
struct part2 = part1; // assigns values of part1 to part2
```

In the next program we have defined two structure data types **date** and **student**. We are using **birthday** as a structure variable inside student structure. The data type of **birthday** is a **date** structure.

*Program 13.2: structure.cpp*

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

struct date   { int month; int day; int year; };
struct student{ char name[80];  char mobile[15]; int age;
                char email[30]; date birthday;            };
int main(){
     student sd1;
     cout << "Please Enter Your Information " << endl;

     cout << "Name: ";             gets(sd1.name);
     cout << "mobile Number: ";    gets(sd1.mobile);
     cout << "Your Age: ";         cin >> sd1.age;
     cout << "Please Enter Your Birthday \n";
     cout << "Year: ";              cin >> sd1.birthday.year;
     cout << "month(1-12): ";       cin >> sd1.birthday.month;
     cout << "Day(1-31): ";         cin >> sd1.birthday.day;

     cout << endl << "Here is Your Information"<< endl;
     cout << "Name    : " << sd1.name   << endl;
     cout << "Mobile  : " << sd1.mobile << endl;
     cout << "Age     : " << sd1.age <<" years old \n";
     cout << "Birthday: "
          << sd1.birthday.day <<"-"
          << sd1.birthday.month <<"-"
          << sd1.birthday.year << endl;

     system( "PAUSE" ); return 0;
}
```

This will be the output of program

```
Please Enter Your Information
Name: Rashid Farid
mobile Number: 0321-53001234
Your Age: 40
Please Enter Your Birthday
Year: 1980
month(1-12): 1
Day(1-31): 1

Here is Your Information
Name    : Rashid Farid
Mobile  : 0321-53001234
Age     : 40 years old
Birthday: 1-1-1980
Press any key to continue . . .
```

## TASK 1:

Write a C++ program that stores information of about a **student** containing following details: Registration Number, Name, E-mail Address, Name of the Department, Current Semester, Credit Hours Registered, and Current CGPA.

- First take this information from a user
- Store this information in a structure
- Show all information on screen in a nice format

## Solution of Task 1:

## Solution of Task 1:

# Lab 14 : Pointers.

Pointers are a special type of variables in which a memory address is stored. They contain a memory address, not the value of the variable. Pointers are an important and essential tool for increasing the power of C++. A notable example is the creation of data structures such as **linked lists** and **binary trees**. In fact, several key features of C++, such as **virtual functions**, the **new operator**, and the **this pointer** require the use of pointers.

The ideas behind pointers are not complicated. Here's the first key concept: Every byte in the computer's memory has an address. Addresses are numbers, just as they are for houses on a street. The numbers start at 0 and go up from there 1, 2, 3, and so on. If you have 1KB of memory, the highest address is 1023. (Of course you have much more.) Your program, when it is loaded into memory, occupies a certain range of these addresses. That means that every variable and every function in your program starts at a particular address. You can find the **address occupied by a variable** by using the **address-of operator &**.

*Program 14.1: address.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main(){
  int    var1  = 123456;            // define and
  double var2  = 4.70495e+018;      // initialize
  char   var3 = 'A';                // three variables
  cout << "Address of Var1 = " << &var1 << endl;
  cout << "Address of Var2 = " << &var2 << endl;
  cout << "Address of Var3 = "
       << static_cast<void *>(&var3)    << endl;
  system("PAUSE");
  return 0;
}
```

This will be the output of program

```
Address of Var1 = 0x6ffe1c
Address of Var2 = 0x6ffe10
Address of Var3 = 0x6ffe0f
```

This will be the memory layout according to the program output. But you may see different addresses after compiling and executing C++ code shown on previous page.

| RAM Address | Data | Variable Name |
|---|---|---|
| 0x6ffe0f | 'A' | Var3 (1 Byte) |
| 0x6ffe10<br>0x6ffe11<br>0x6ffe12<br>0x6ffe13<br>0x6ffe14<br>0x6ffe15<br>0x6ffe16<br>0x6ffe17 | 4.70495e+018 | Var2 (8 Bytes) |
| 0x6ffe18 | | Unused Memory |
| 0x6ffe19 | | |
| 0x6ffe1a | | |
| 0x6ffe1b | | |
| 0x6ffe1c<br>0x6ffe1d<br>0x6ffe1e<br>0x6ffe1f | 123456 | Var1 (4 Bytes) |
| 0x6ffe20 | | Unused Memory |

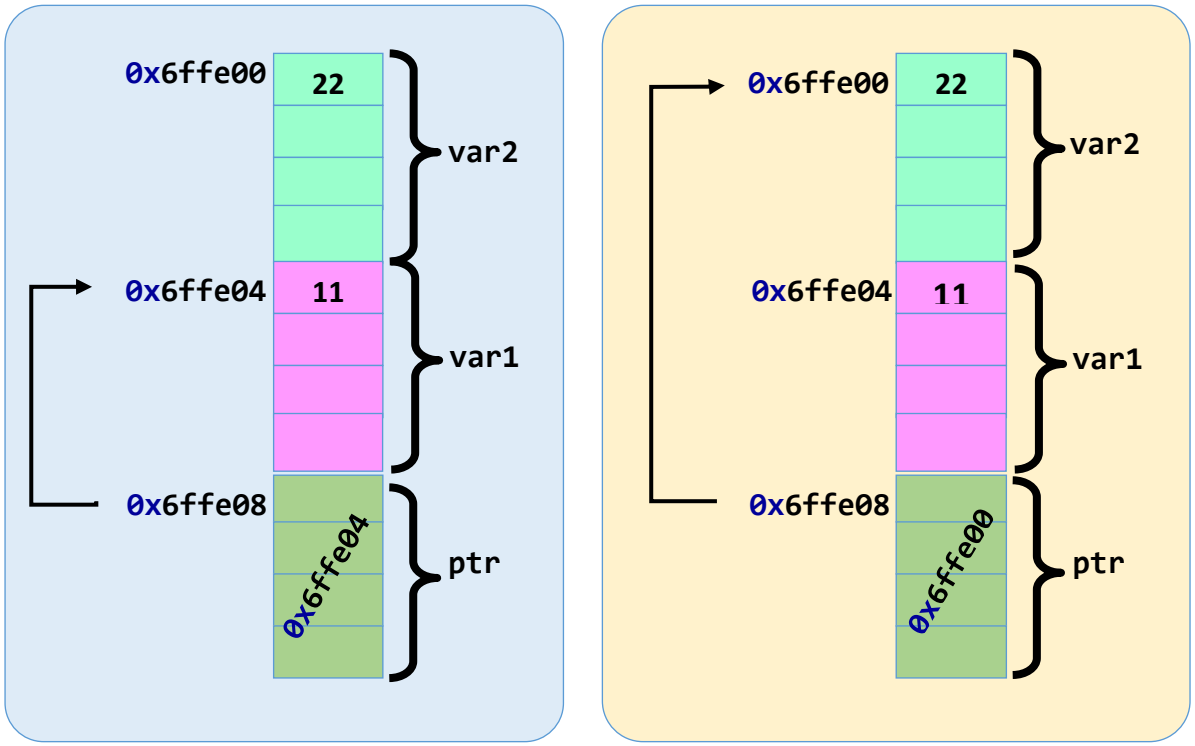
*Program 14.2:pointer.cpp*

```
#include <iostream>
#include <stdlib.h>
using namespace std;
int main() {
 int *ptr; int var1 = 11, var2 = 22;
 cout << "& ptr  = " << &ptr << endl;
 cout << "& var1 = " << &var1 << endl;
 cout << "& var2 = " << &var2 << endl;
 ptr = &var1;                          // pointer points to var1
 cout <<"  ptr  = " <<  ptr << endl;   // print pointer value
 cout <<" *ptr  = " << *ptr << endl;   // print value of var1
 ptr = &var2;                          // pointer points to var2
 cout <<"  ptr  = " <<  ptr << endl;   // print pointer value
 cout <<" *ptr  = " << *ptr << endl;   // print value of var2
 system("PAUSE"); return 0;
}
```

This will be the output of program

```
ptr   = 0x6ffe08
& var1 = 0x6ffe04
& var2 = 0x6ffe00
  ptr   = 0x6ffe04
 *ptr   = 11
  ptr   = 0x6ffe00
 *ptr   = 22
Press any key to continue . . .
```

In the program 14.2, we have created one integer pointer and two integer variables. First the pointer **ptr** takes the address of variable **var1** and then it takes the address of variable **var2**.




## TASK 1:

Initialize an integer array of 5 elements. Display values of all elements along with their addresses using pointers.

## TASK 2:

Initialize a char pointer with some text. e.g. `char* p = "Hello World";`

Declare a char array of size 50. Copy the statement from character pointer **p** to the character array including null character.

## Solution of Task 1:

## Solution of Task 2:

# Lab 15 File Handling.

A **file** is a collection of related data. C treats a file as a series of bytes. Many files reside on hard disk or usb memory stick. The declarations for the structures and functions used by the file functions are stored in the standard include file **`<stdio.h>`**

Before doing anything with files, you must put the line: **`#include <stdio.h>`**

At the beginning of your program. The declaration for a file variable **fpt** is: **`FILE *fpt;`**

Before a file can be used, it must be **opened** using the function **`fopen()`**. fopen() returns a pointer to the file structure for the file.

The format for fopen() is: **`fpt = fopen(name, mode);`**

Where,

- **fpt**: is a file pointer. A value of **NULL** is returned on error.
- **name**: is the actual name of the file **data.txt** or **temp.dat** etc.
- **mode**: indicates if the file is to be read or written.
    - mode is "w" for writing and
    - "r" for reading.
    - The flag "b" can be added to indicate a **binary file**.
    - Omitting the binary flag indicates an **ASCII (text) file**.
    - Flags can be combined. So **"wb"** is used to write a binary file and So **"rb"** is used to read a **binary file**. The function returns a **file handle** that will be used in subsequent I/O operations.

If there is an I/O error, then the value **NULL** is returned:

The function **`fclose()`** will close the file. The format of fclose() is:

**`fclose(fpt);`** OR **`status = fclose(fpt);`**

- The variable **status** is zero if the **`fclose()`** was successful or nonzero for an error.

## Writing to A file:

*Program 15.1: fputc.cpp*

```
#include <stdio.h> // for FILE and Functions
#include <ctype.h> // for toupper()
int main( ){
     FILE * fpt ; // define a structure File pointer
     char c;      // define a variable c to store a character
```

```
      // open a new data file named data.txt for writing only
      fpt = fopen("data.txt", "w");
      // read each character from user and write its uppercase
      // equivalent to the data file until the user gives Enter.
      while(true){
            c = getchar();  // Read a character from user
            if(c == '\n')   // If it is Enter key
                  break;    // break the loop
            c = toupper(c); // break the loop
            putc(c, fpt );  // Write on Byte on a File
      }
      fclose ( fpt ) ;      // close the data file
      system("PAUSE");
      return 0;
}
```

Reading from A file:

*Program 15.2: fgetc.cpp*

```
#include <stdio.h> // for FILE structure
#include <iostream> // for cin cout
using namespace std;
int main( ){
      FILE * fpt ;    // define a FILE pointer
      char c;

      // open a data.txt file for reading only
      fpt = fopen("data.txt", "r");
      if ( fpt == NULL ){
            cout << "cannot open file \n";
            system("PAUSE");
            exit(0) ;
      }
```

```
      while(true) {
            c = fgetc(fpt);  // read one character from a file
            if(c == EOF)    // if (c == -1)
                  break;     // break the loop
            cout << c;       // show the character on screen
      }
      fclose ( fpt ) ;       // close the data file
      system("PAUSE");       return 0;
}
```

## Writing and Reading File Using Structure

*Program 15.3: person.cpp*

```
#include <stdio.h>
#include <conio.h>
#include <iostream>
using namespace std;
void Add_Record();
void View_All_Records();
struct PERSON{
      char name[50] ; int  age;
};
int main( ){
      char option;
      cout <<"Press 1 to Add Record and 2 to View All Records: ";
      cin >> option; fflush(stdin);
      if(option=='1')
            Add_Record();
      else
            View_All_Records();
      system("PAUSE");
      return 0;
}
```

```
void Add_Record(){
      struct PERSON P1;
      char choice;
      FILE* fp = fopen ( "address_book.txt", "wb" ) ;
      if ( fp  == NULL ){
            cout << "Can not open file \n" ;
            system("PAUSE"); exit(0) ;
      }
      do{
            cout << "Enter Name: "; cin.get(P1.name,50);
            cout << "Enter Age : "; cin >> P1.age;
            fwrite ( &P1, sizeof ( P1 ), 1, fp ) ;
            cout << "\n Add another record (Y/N) " ;
            cin >> choice;
            fflush (stdin) ; // fflush (stdout);
      } while (choice == 'y'|| choice == 'Y');
      fclose ( fp ) ;
}

void View_All_Records(){
      struct PERSON P1;
      FILE* fp = fopen ( "address_book.txt", "rb" ) ;
      if (  fp == NULL ){
            cout << "Can not open file \n" ;
            system("PAUSE"); exit(0) ;
      }
      while ( fread ( &P1, sizeof ( P1 ), 1, fp ) == 1 ){
            cout << P1.name <<" is "<<P1.age <<" years old"
                  << endl;
      }
      fclose ( fp ) ;
}
```

## TASK 1:

Write name, age and CNIC No. of a person into a data file “person.txt” and then read file and show details of all persons you have entered in the program.

## Solution of Task 1: